

SIMATS ENGINEERING

ASSESSMENT TOOL 2 (Medium)- Debugging and Optimization

COURSE NAME: Data Structures

COURSE CODE:CSA0316

COURSE OUTCOME COVERED

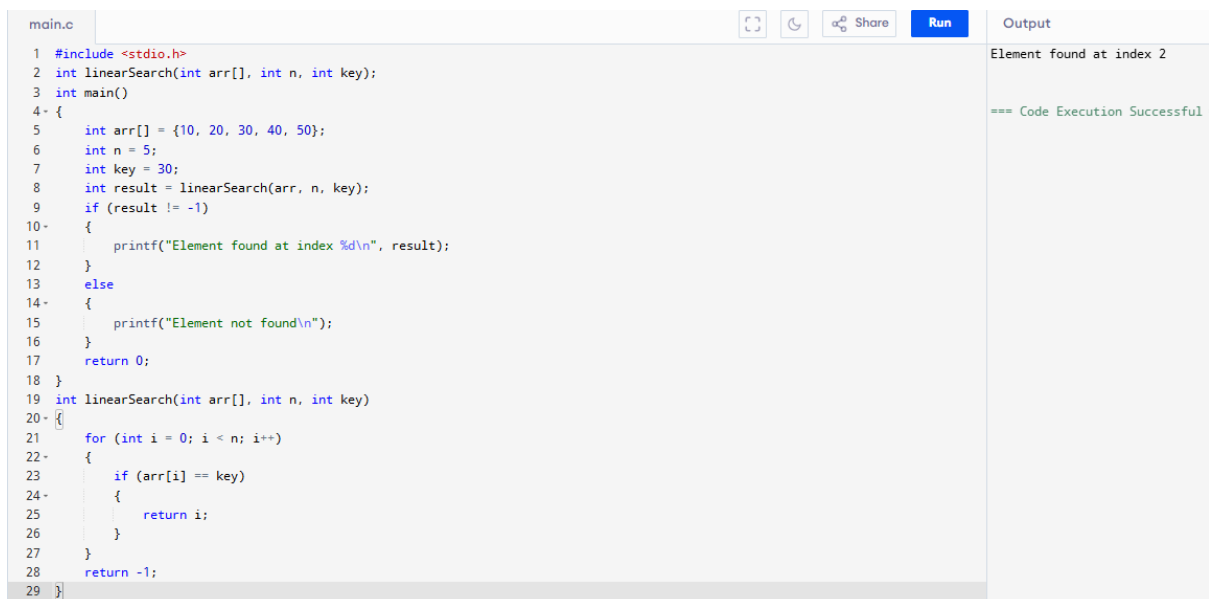
CO2 : Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques. (BTL3, BTL4)

Assessment Tool Weightage: **10%**

1. Find the bugs, include the missing lines in the following snippet.

```
int linearSearch(int arr[], int n, int key) {  
  
for(int i = 0; i <= n; i++) {  
  
if(arr[i] == key)  
  
return i;  
  
}  
  
return -1;  
  
}
```

ANS:



The screenshot shows a C++ IDE with a file named 'main.c'. The code defines a function 'linearSearch' and a 'main' function. The 'linearSearch' function has a bug: it uses 'i <= n' in the for loop, which is incorrect as it should be 'i < n'. The 'main' function calls 'linearSearch' with an array {10, 20, 30, 40, 50}, n=5, and key=30. The output shows 'Element found at index 2' and 'Code Execution Successful'.

```
1 #include <stdio.h>  
2 int linearSearch(int arr[], int n, int key);  
3 int main()  
4 {  
5     int arr[] = {10, 20, 30, 40, 50};  
6     int n = 5;  
7     int key = 30;  
8     int result = linearSearch(arr, n, key);  
9     if (result != -1)  
10    {  
11        printf("Element found at index %d\n", result);  
12    }  
13    else  
14    {  
15        printf("Element not found\n");  
16    }  
17    return 0;  
18 }  
19 int linearSearch(int arr[], int n, int key)  
20 {  
21     for (int i = 0; i < n; i++)  
22     {  
23         if (arr[i] == key)  
24         {  
25             return i;  
26         }  
27     }  
28     return -1;  
29 }
```

Output
Element found at index 2
=== Code Execution Successful ===

2. Buggy Code in an array

```
#include <stdio.h>

int main() {

    int arr[5] = {10,20,30,40,50};

    for(int i=0;i<=5;i++)

        printf("%d ",arr[i]);

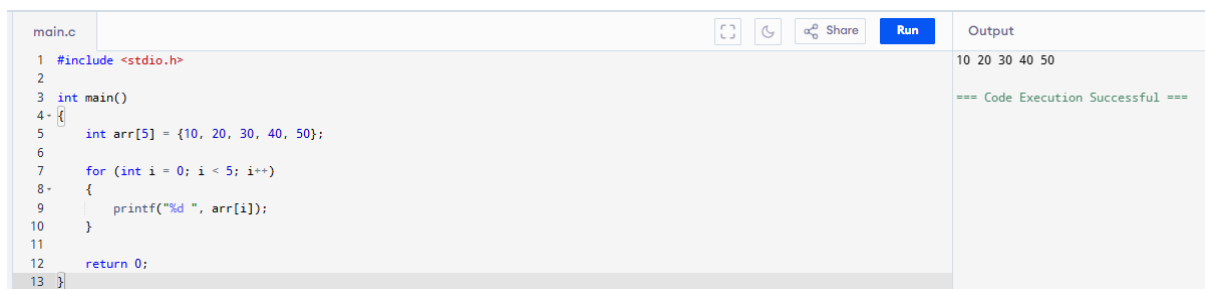
    return 0;

}
```

Expected output:

10 20 30 40 50

ANS:



The screenshot shows a code editor with a file named 'main.c'. The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int arr[5] = {10, 20, 30, 40, 50};
6
7     for (int i = 0; i < 5; i++)
8     {
9         printf("%d ", arr[i]);
10    }
11
12    return 0;
13 }
```

On the right side, there is an 'Output' panel. It displays the output '10 20 30 40 50' and a status message '=== Code Execution Successful ==='. The code editor also has icons for running, sharing, and other functions.

3. Identify an errors in the following snippet

```
int binarySearch(int arr[], int n, int key)
```

```
{
```

```
    int low = 0, high = n - 1;
```

```
    while(low < high)
```

```
    { int mid = (low + high) / 2;
```

```
      if(arr[mid] == key) return mid;
```

```
      else if(arr[mid] < key)
```

```
      low = mid;
```

```
      else high = mid;
```

```

}

return -1;

}

```

ANS:

The screenshot shows a C code editor with a file named 'main.c'. The code implements a binary search function. The main function initializes an array {10, 20, 30, 40, 50} with n=5 and a key=30. It calls the binarySearch function, which returns 2. The main function then prints 'Element found at index 2'. The output panel on the right shows 'Element found at index 2' and '=== Code Execution Successful ==='.

```

1 #include <stdio.h>
2 int binarySearch(int arr[], int n, int key);
3 int main()
4 {
5     int arr[] = {10, 20, 30, 40, 50};
6     int n = 5;
7     int key = 30;
8
9     int result = binarySearch(arr, n, key);
10
11     if (result != -1)
12     {
13         printf("Element found at index %d\n", result);
14     }
15     else
16     {
17         printf("Element not found\n");
18     }
19
20     return 0;
21 }
22 int binarySearch(int arr[], int n, int key)
23 {
24     int low = 0, high = n - 1;
25
26     while (low <= high)
27     {
28         int mid = (low + high) / 2;
29
30         if (arr[mid] == key)
31         {
32             return mid;
33         }
34         else if (arr[mid] < key)
35         {
36             low = mid + 1;
37         }
38         else
39         {
40             high = mid - 1;
41         }
42     }
43     return -1;
44 }

```

Output: Element found at index 2
=== Code Execution Successful ===

4. Buggy Code in Stack Push Operation

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```

void push(int data) {
    if (top == MAX - 1)
        printf("Stack Overflow");
    else
        stack[top++] = data;
}

```

```

int main() {
    push(10);
    push(20);
}

```

```
    printf("%d", stack[top]);  
}
```

5. Buggy Code in Stack POP Operation

```
#include <stdio.h>  
#define MAX 5  
int stack[MAX];  
int top = 2;  
int pop() {  
    if (top == -1)  
        return -1;  
    return stack[++top];  
}
```

ANS:

main.c	Output
<pre>1 #include <stdio.h> 2 #define MAX 5 3 int stack[MAX]; 4 int top = -1; 5 void push(int data); 6 int main() 7 { 8 push(10); 9 push(20); 10 11 printf("%d", stack[top]); 12 13 return 0; 14 } 15 void push(int data) 16 { 17 if (top == MAX - 1) 18 { 19 printf("Stack Overflow"); 20 } 21 else 22 { 23 stack[++top] = data; 24 } 25 }</pre>	<pre>20 === Code Execution Successful ===</pre>

6. Buggy Code in underflow operation

```
int pop() {  
    if(top == 0) {  
        printf("Underflow");  
        return -1;  
    }  
    return stack[top--];  
}
```

ANS:

```
main.c
1 #include <stdio.h>
2 #define MAX 5
3 int stack[MAX];
4 int top = -1;
5 int pop(void);
6 int main()
7 {
8     stack[++top] = 10;
9     stack[++top] = 20;
10
11     printf("Popped element: %d\n", pop());
12
13     return 0;
14 }
15 int pop(void)
16 {
17     if (top == -1)
18     {
19         printf("Stack Underflow\n");
20         return -1;
21     }
22
23     return stack[top--];
24 }
25
```

Output

Popped element: 20

=== Code Execution Successful ===

7. Find the issue in the following code snippet.

```
#include<stdio.h>
```

```
#define MAX 5
```

```
void main()
```

```
{
```

```
int queue[MAX],x;
```

```
int front = 0, rear = 0;
```

```
printf("Enter the value");
```

```
scanf("%d",&x);
```

```
void enqueue(int x) {
```

```
    if(rear == MAX) {
```

```
        printf("Overflow");
```

```
        return;
```

```
    }
```

```
    else{
```

```
        queue[rear++] = x;
```

```
        printf("Value is %d", queue[rear]);
```

```
    }}}
```

ANS:

```
main.c
1 #include <stdio.h>
2 #define MAX 5
3 int queue[MAX];
4 int front = 0, rear = 0;
5 void enqueue(int x);
6 int main()
7 {
8     int x;
9
10    printf("Enter the value: ");
11    scanf("%d", &x);
12
13    enqueue(x);
14
15    return 0;
16 }
17 void enqueue(int x)
18 {
19     if (rear == MAX)
20     {
21         printf("Queue Overflow\n");
22         return;
23     }
24     else
25     {
26         queue[rear++] = x;
27         printf("Value is %d\n", queue[rear - 1]);
28     }
29 }
```

Output

Enter the value: 25
Value is 25

=== Code Execution Successful ===

8. Buggy code in node creation

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node *next;
};
```

```
int main() {
    struct Node *newNode;

    newNode->data = 10;
    newNode->next = NULL;

    printf("%d", newNode->data);

    return 0;
}
```

ANS:

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 struct Node
4 {
5     int data;
6     struct Node *next;
7 };
8 int main()
9 {
10    struct Node *newNode;
11    newNode = (struct Node *)malloc(sizeof(struct Node));
12    newNode->data = 10;
13    newNode->next = NULL;
14    printf("%d\n", newNode->data);
15    free(newNode);
16    return 0;
17 }
```

Output

10

=== Code Execution Successful ===

9. Include missing statements and display the list of elements in linkedlist

```
void display(struct Node *head)
{
while(head != NULL);
{
printf("%d ", head->data);
head = head->next;
}}
```

ANS:

<pre>main.c 1 #include <stdio.h> 2 #include <stdlib.h> 3 struct Node 4 { 5 int data; 6 struct Node *next; 7 }; 8 void display(struct Node *head); 9 int main() 10 { 11 struct Node node1, node2; 12 13 node1.data = 10; 14 node1.next = &node2; 15 16 node2.data = 20; 17 node2.next = NULL; 18 19 display(&node1); 20 21 return 0; 22 } 23 void display(struct Node *head) 24 { 25 while (head != NULL) 26 { 27 printf("%d ", head->data); 28 head = head->next; 29 } 30 }</pre>	Output 10 20 === Code Execution Successful ===
---	---

10. Bug in Queue Full Condition-Identify and fix the errors.

```
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int item)
{
    if(rear == SIZE-1)
    {
        printf("Queue Full");
        return;
    }

    if(front == -1)
        front = 0;

    rear = (rear + 1) % SIZE;
    queue[rear] = item;
}
```

main.c

Share

Run

Output

```

1 #include <stdio.h>
2 #define SIZE 5
3 int queue[SIZE];
4 int front = -1, rear = -1;
5 void enqueue(int item);
6 int main()
7 {
8     enqueue(10);
9     enqueue(20);
10    printf("Front element: %d\n", queue[front]);
11
12    return 0;
13 }
14 void enqueue(int item)
15 {
16     if ((rear + 1) % SIZE == front)
17     {
18         printf("Queue Full\n");
19         return;
20     }
21
22     if (front == -1)
23     {
24         front = 0;
25     }
26
27     rear = (rear + 1) % SIZE;
28     queue[rear] = item;
29 }

```

Front element: 10

=== Code Execution Successful ===

Code of Conduct:

I K.Pavithra(192472155) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:K.Pavithra

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues

Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation